

Técnicas de Programación Concurrente I

Resumen completo — v2 (con práctica y problemas clásicos)

Ing. Pablo A. Deymonnaz · FIUBA · Depto. de Computación

Incluye: Rust esencial · Vectorización · Condvar · Problemas clásicos completos · Chandy-Misra

Unidad	Tema
0	Rust Esencial para Concurrency
1	Introducción — Definiciones y Modelos
2	Modelo Fork-Join y Vectorización
3	Programación Asíncrona
4	Corrección y Locks
5A	Sincronización — Semáforos
5B	Condvar (Variables de Condición)
5C	Monitores
6	Redes de Petri
7A	Pasaje de Mensajes / Channels
7B	Modelo de Actores
★	Problemas Clásicos (Pseudocódigo completo)
✓	Checklist rápido pre-parcial

0. Rust Esencial para Concurrencia

¿Por qué Rust?

- **Seguridad de memoria** en tiempo de compilación (sin GC): previene buffer overflow, dangling pointers, use-after-free.
- **Zero-cost abstractions**: abstracciones de alto nivel sin overhead en runtime.
- **Concurrencia sin miedo**: el compilador previene condiciones de carrera en tiempo de compilación.

Ownership — Reglas fundamentales

- Cada valor tiene exactamente **un dueño (owner)** a la vez.
- Cambiar de dueño = hacer un **move**. El dueño original pierde el acceso.
- El dueño puede prestar: **múltiples referencias inmutables** (&T;) O **una única referencia mutable** (&mut; T), nunca ambas simultáneamente.
- Cuando el dueño sale de scope, el valor es **dropped** (trait Drop = RAII).

```
fn envejecer(mut p: Persona) { p.edad += 20; } // toma ownership → ariel no existe más
fn envejecer(p: &mut Persona) { p.edad += 20; } // presta ref mutable → ariel sigue vivo
```

Smart Pointers para concurrencia

Tipo	Para qué	Thread-safe
Box<T>	Dato en el heap, único dueño	No aplica
Rc<T>	Múltiples referencias inmutables, mismo thread	No
Arc<T>	Múltiples referencias inmutables, múltiples threads (ref counting atómico)	Sí
Mutex<T>	Exclusión mutua para T (write lock)	Sí (con Arc)
RwLock<T>	Múltiples lectores O un escritor	Sí (con Arc)

Arc + RwLock — Patrón típico

```
use std::sync::{Arc, RwLock}; use std::thread; let data = Arc::new(RwLock::new(0i64)); let
data_clone = Arc::clone(&data); let handle = thread::spawn(move || { let mut w =
data_clone.write().unwrap(); // exclusive lock *w += 1; }); // lock liberado aquí por RAII (Drop)
handle.join().unwrap(); println!("{}", *data.read().unwrap()); // shared lock
```

Traits Send y Sync

- **Send**: el ownership del tipo puede transferirse entre threads. Casi todos los tipos son Send. Excepción: Rc<T>.
 - **Sync**: es seguro referenciar desde múltiples threads. T es Sync si &T; es Send. Excepción: cell<T>, RefCell<T>.
 - Los tipos compuestos por tipos Send/Sync automáticamente implementan Send/Sync.
- Si intentás compartir algo que no es Send/Sync entre threads, el compilador te frena en tiempo de compilación.

Option y Result — manejo de errores

- `Option<T>`: `Some(T)` o `None`. Reemplaza `null`.
- `Result<T, E>`: `Ok(T)` o `Err(E)`. Para operaciones que pueden fallar.
- `.unwrap()` → `panic` si es `None/Err`. `.expect(msg)` → igual pero con mensaje. `?` → propaga el error.

1. Introducción — Definiciones y Modelos de Concurrency

Definiciones clave

Término	Definición
Programa	Instrucciones que se ejecutan secuencialmente en un procesador.
Proceso	Programa secuencial compuesto por instrucciones atómicas.
Programa concurrente	Conjunto finito de procesos secuenciales que pueden ejecutarse en paralelo. Ejecuta el intercalado arbitrario de instrucciones atómicas.
Instrucción atómica	Se ejecuta de principio a fin sin interrupciones, o no se ejecuta.
Paralelo	Las ejecuciones se superponen en tiempo en procesadores distintos.
Concurrente	Paralelismo potencial: las ejecuciones pueden superponerse.
Multitasking	Ejecución de múltiples procesos concurrentemente; el scheduler del SO coordina el acceso a los procesadores.
Multithreading	Feature del lenguaje: ejecución concurrente de más de un thread dentro de un programa.
Thread	Comparte recursos del proceso (espacio de memoria). Mantiene su propio stack, PC y registros.

Modelos de concurrency

- **Estado mutable compartido** — requiere serializar acceso (locks, semáforos, monitores).
- **Fork-Join** — tareas independientes en paralelo, resultados se unen.
- **Canales / Mensajes** — procesos se comunican enviando mensajes.
- **Programación asíncrona** — tareas livianas que ceden el control al esperar I/O.
- **Actores** — actores aislados que solo se comunican por mensajes asíncronos.

2. Modelo Fork-Join y Vectorización

Fork-Join

El cómputo (task) se parte recursivamente en sub-cómputos independientes (subtasks). Los resultados parciales se unen (join) para construir la solución final.

- Las tareas son **independientes** → se pueden ejecutar en paralelo.
- Tareas **no se bloquean** excepto para esperar el fin de sus subtareas.
- **Determinístico**: resultado siempre igual sin importar velocidad de threads.
- **Sin condiciones de carrera** porque los threads están aislados.
- **Performance ideal**: $t_{\text{secuencial}} / N_{\text{threads}}$.
- **Desventaja**: requiere que las unidades de trabajo estén aisladas.

Work Stealing

Algoritmo de scheduling. Worker threads inactivos roban trabajo a threads ocupados para balancear la carga.

- Cada thread tiene su propia **deque** (cola doble) con tareas listas.
- Al terminar tarea: agrega subtareas al **final** de su deque.
- Toma siguiente tarea del **final** de su deque (*LIFO propio*).
- Si deque vacía: **roba** del **inicio** de la deque de otro thread aleatorio (*FIFO ajeno*).
- Ventaja: baja sincronización, los workers solo se comunican cuando lo necesitan.

Rayon y Crossbeam en Rust

- **Rayon**: `rayon::join(f1, f2)` — dos tareas en paralelo. `.par_iter()` — itera en paralelo. Implementa work stealing internamente.
- **Crossbeam**: `crossbeam::scope` — garantiza que todos los threads terminan antes de salir del scope. Retorna Ok o Err (si alguno hizo panic).

```
// Rayon: map-reduce paralelo let result = data.par_iter().map(|x| process(x)).reduce(|| identity(), |a, b| merge(a, b));
```

Vectorización (SIMD)

Caso especial de paralelización **automática**. Al estancarse la Ley de Moore, se agregaron múltiples ALUs para operar sobre los mismos registros de forma concurrente → juegos de instrucciones SIMD (*Single Instruction, Multiple Data*): MMX, SSE, AVX (x86), NEON (ARM).

- Procesa una operación en múltiples pares de operandos **a la vez**.
- Cada variable (registro) es un **vector de tamaño fijo** (generalmente 128-512 bits).
- Se divide en N **lanes** (carriles): un registro de 512 bits puede alojar 16 enteros de 32 bits, o 8 flotantes de 64 bits.
- Casos de uso: sonido, imágenes, video, redes neuronales. Cada dato es independiente.

Operación	Descripción	Velocidad
Vertical	Entre distintos registros en el mismo lane. Ej: sumar vector A + vector B lane a lane.	Rápidas (paralelas)
Horizontal	En el mismo registro, entre distintos lanes. Reducen un vector a un escalar. Ej: $x_0 + x_1 + x_2 + x_3$.	Lentas (secuenciales)

3. Programación Asíncrona

¿Cuándo usar async?

Usar async	NO usar async (usar threads/sync)
Consultar servicios externos (HTTP, DB)	Calcular un factorial
Leer/escribir archivos	Multiplicar matrices
Servir múltiples requests concurrentes	Implementar filósofos (estado compartido)
Miles de conexiones I/O-bound	Cómputo CPU-intensivo puro

Futures

El trait `std::future::Future` representa una operación sobre la que se puede testear si se completó. El método `poll` **nunca bloquea**.

- Si completó → `Poll::Ready(output)`. Si no → `Poll::Pending`.
- Modelo **piñata**: lo único que se puede hacer con un future es hacer `poll` hasta que retorne el valor.
- Cada `poll` avanza todo lo que puede y almacena su estado para el siguiente `poll`.

```
trait Future { type Output; fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>; } enum Poll<T> { Ready(T), Pending }
```

Pin

Trait implementado por los Futures para garantizar que su referencia a sí mismos no sea movida en memoria una vez que se llamó a `poll()`. Evita self-references inválidas.

async fn y await

- Invocar una función `async` retorna **inmediatamente** un Future (sin ejecutar el cuerpo).
- Primer `poll`: ejecuta el cuerpo hasta el primer `await`.
- `await` toma ownership del future anidado y hace `poll`. Si `Pending` → retorna `Pending` al caller.
- El cambio de una tarea a otra ocurre **solo en los puntos await**.
- Un cómputo grande sin `await` no da lugar a otras tareas (diferencia clave con threads).

Executors

Función	Descripción
<code>block_on</code>	Función sincrónica que espera el resultado de una función <code>async</code> . Adaptador <code>sync↔async</code> . No usar dentro de código <code>async</code> (bloquea el thread).
<code>spawn_local</code>	Agrega future al pool que realiza polling en <code>block_on</code> . Lifetimes deben ser 'static. Todo en un único thread.
<code>task::spawn</code>	Crea la tarea en el pool de threads dedicado. No necesita <code>block_on</code> para ser polleada.
<code>yield_now</code>	Favorece el paralelismo: retorna <code>Pending</code> la primera vez, <code>Ready</code> la siguiente. Cede el control a otras tareas.

spawn_blocking	Para cálculos pesados/bloqueantes: coloca la tarea en otro thread del SO.
----------------	---

Concurrencia colaborativa: el thread duerme hasta que haya algo que pollear. La eficiencia viene de que el SO no necesita hacer context switch por interrupción, sino que el código mismo cede el control voluntariamente.

4. Corrección y Locks

Propiedades de corrección

Tipo	Propiedad	Descripción
Safety	Exclusión mutua	Dos procesos no deben intercalar instrucciones de su sección crítica.
Safety	Ausencia de deadlock	Un sistema no finalizado debe poder continuar avanzando productivamente.
Liveness	Ausencia de starvation	Todo proceso listo para usar un recurso debe recibirlo eventualmente.
Liveness	Fairness	Una instrucción continuamente habilitada eventualmente aparece en el escenario.

Sección Crítica

Cada proceso ejecuta un loop infinito con parte crítica y parte no crítica.

- La sección crítica debe **progresar** (finalizar eventualmente).
- La sección no-crítica no requiere progreso.
- Especificaciones: Exclusión mutua + Ausencia de deadlock + Ausencia de starvation.

Locks (RwLock)

- **Shared lock (lectura)**: múltiples procesos pueden tenerlo simultáneamente. Para tomarlo: esperar a que se liberen todos los exclusive locks.
- **Exclusive lock (escritura)**: solo un proceso. Para tomarlo: esperar a que se liberen todos los locks (de ambos tipos).
- **Lock envenenado**: un thread toma write lock y hace panic!. Llamadas posteriores a read()/write() retornan Error.

```
use std::sync::{Arc, RwLock}; let lock = Arc::new(RwLock::new(5i32)); // Lectura compartida let r = lock.read().unwrap(); // bloquea si hay writer println!("{}", *r); // 'r' (guard) liberado aquí por RAII // Escritura exclusiva let mut w = lock.write().unwrap(); // bloquea si hay readers o writers *w += 1;
```


5A. Semáforos

Definición y operaciones

Tipo de dato: entero no negativo V + conjunto de procesos L . Inicializa con $k \geq 0$ y $L = \emptyset$. **Si $V > 0$** : recurso disponible. **Si $V = 0$** : no disponible.

Operación	Efecto
<code>wait(S) / p(S)</code>	Si $V > 0$: $V \leftarrow V - 1$. Si no: proceso bloqueado, se agrega a L .
<code>signal(S) / v(S)</code>	Si L vacío: $V \leftarrow V + 1$. Si no: saca proceso q de L y lo pone ready.

- **Invariante:** $S.V = k + \# \text{signal}(S) - \# \text{wait}(S) \geq 0$.
- **Semáforo binario / Mutex:** $V \in \{0, 1\}$. Equivale a un write lock.
- `wait()` y `signal()` son **instrucciones atómicas**.
- `signal()` despierta un proceso **arbitrario** (no determinístico quién).

Semáforos en Rust — `crate std::semaphore`

```
let sem = Semaphore::new(5); sem.acquire(); // wait - decrementa sem.release(); // signal - incrementa
let _guard = sem.access(); // wait con RAI (release automático al salir de scope)
```

Barreras — `std::sync::Barrier`

- `Barrier::new(n)` — sincroniza n threads en un punto.
- `barrier.wait()` — bloquea hasta que todos lleguen. Retorna `BarrierWaitResult`.
- `is_leader()` — true solo en el último thread que llama a `wait` (desbloquea al resto).
- Las barreras son **reutilizables** automáticamente.

5B. Variables de Condición (Condvar)

Definición

Mecanismo de sincronización que permite a un thread esperar hasta que cierta condición sea verdadera, **liberando el Mutex mientras espera**.

- No guarda ningún valor.
- Tiene asociado un FIFO de procesos esperando.
- Siempre se usa **junto con un Mutex** (patrón Arc<(Mutex, Condvar)>).

Operaciones (teóricas)

Operación	Efecto
waitC(cond)	Siempre bloquea al proceso. Libera el lock del monitor automáticamente. Agrega el proceso al FIFO.
signalC(cond)	Si hay procesos en el FIFO: saca el del tope y lo pone ready. Si no hay: no hace nada.
empty(cond)	Retorna true si no hay procesos esperando.

Condvar en Rust — std::sync::Condvar

```
use std::sync::{Arc, Mutex, Condvar}; let pair = Arc::new((Mutex::new(false), Condvar::new())); //
Waiter thread let (lock, cvar) = &*pair; let mut started = lock.lock().unwrap(); while !*started { //
<-- loop para manejar spurious wakeups started = cvar.wait(started).unwrap(); } // Notifier thread
let (lock, cvar) = &*pair; *lock.lock().unwrap() = true; cvar.notify_one(); // o notify_all()
```

■ **Siempre usar while, NO if, para verificar la condición. Los spurious wakeups (despertar sin que nadie haya notificado) son posibles en cualquier implementación de condvars.**

- wait_while(guard, |state| condicion) — forma idiomática en Rust: espera mientras la condición sea true.
- notify_one() — despierta un waiter arbitrario.
- notify_all() — despierta todos los waiters (útil cuando múltiples threads pueden avanzar).

Semáforo vs Monitor (Condvar)

Semáforo	Monitor / Condvar
wait puede o no bloquear	waitC siempre bloquea
signal siempre tiene efecto (V++)	signalC no tiene efecto si cola vacía
signal desbloquea proceso arbitrario	signalC desbloquea el tope de la cola FIFO
Proceso desbloqueado continúa inmediatamente	Proceso desbloqueado espera que el señalizador deje el monitor

5C. Monitores

Definición

Herramienta de sincronización que provee exclusión mutua implícita y condvars. Solo un proceso puede ejecutar el monitor a la vez.

Componentes

- Nombre, variables internas, procedimientos (acceden a variables internas), interfaz pública, inicialización, condition variables.

Estados de proceso en el monitor

- Esperando para entrar al monitor.
- Ejecutando el monitor (solo un proceso — exclusión mutua).
- Bloqueado en FIFO de condition variable.
- Recién liberado de waitC.
- Recién completó una operación signalC.

Monitores en Java — synchronized

- **Bloque synchronized:** `synchronized(obj) {{ ... }}` — el objeto es el lock (monitor).
- **Método synchronized:** el lock es `this` (o la clase para `static`).
- El lock es **reentrante**: un thread puede re-entrar en el mismo bloque `synchronized`.
- `wait()` — libera el monitor y suspende el hilo hasta `notify/notifyAll`.
- `notify()` — despierta un hilo arbitrario que espera por el monitor.
- `notifyAll()` — despierta todos los hilos que esperan.
- `volatile` — la variable siempre se lee de memoria principal (no cachea). No hace locking.

6. Redes de Petri

Definiciones

Herramienta matemática y visual para modelar concurrencia y sincronización.

Red	Definición formal
Ordinaria	$PN = (T, P, A)$ — T : transiciones, P : lugares, $A \subseteq (T \times P) \cup (P \times T)$: arcos
General	$PN = (T, P, A, W, M_0)$ — agrega $W: A \rightarrow \mathbb{N}$ (peso arcos), $M_0: P \rightarrow \mathbb{N} \cup \{0\}$ (marca inicial)

Reglas de disparo

- Transición t **habilitada** $\iff M(p) \geq W(p,t)$ para todo $p \in I(t)$.
- Al disparar: $\forall p \in I(t): M(p) \leftarrow M(p) - W(p,t)$ (consume tokens de entradas)
- $\forall p' \in O(t): M(p') \leftarrow M(p') + W(p',t)$ (produce tokens en salidas)
- **Función de marca:** $M: P \rightarrow \mathbb{N} \cup \{0\}$ — tokens en cada lugar.
- $I(t)$ = lugares de entrada de t . $O(t)$ = lugares de salida de t .

Lugares entrada	Transiciones	Lugares salida
Precondiciones	Eventos	Postcondiciones
Datos de entrada	Cómputos	Datos de salida
Buffers de entrada	Procesador	Buffers de salida

Arco inhibidor

Extensión de la Red General. Un arco inhibidor de P a T hace que la transición T esté habilitada solo si $M(p) = 0$ (el lugar está vacío). Se representa con un círculo en lugar de flecha. Permite modelar **ausencia de tokens**.

- Caso de uso: Lector-Escritor — la transición 'comenzar a leer' está inhibida si hay un escritor.

7A. Pasaje de Mensajes y Canales

Modelos de comunicación

Dimensión	Opciones
Comunicación	Sincrónica (emisor espera al receptor) / Asíncrona con buffer
Direccionamiento	Simétrico (ambos se conocen) / Asimétrico / Sin direccionamiento (por estructura del mensaje)
Flujo de datos	Unidireccional / Bidireccional

Canales en Rust — `std::sync::mpsc`

■ "Do not communicate by sharing memory; instead, share memory by communicating."

- `mpsc` = **M**ultiple **P**roducer, **S**ingle **C**onsumer.
- Transfieren **ownership** del elemento enviado (garantía del borrow checker).
- Para múltiples productores: clonar el transmisor `tx.clone()`.

```
use std::sync::mpsc; let (tx, rx) = mpsc::channel(); // sin buffer (sincrónico) let (tx, rx) = mpsc::sync_channel(10); // con buffer de 10 (asincrónico hasta llenarse) tx.send(valor).unwrap(); // transfiere ownership rx.recv().unwrap(); // bloquea hasta recibir rx.try_recv(); // no bloquea, retorna Result
```

Selective Input

Sintaxis para escuchar en varios canales de forma bloqueante, desbloqueándose con el primero que recibe mensaje.

```
either ch1 => var1 or ch2 => var2
```

Canales en Unix

- **Pipes:** orientados a bytes. Sin representación en filesystem.
- **FIFOs:** como pipes, con representación en filesystem (nombre).
- **Message Queues:** mensajes como unidades independientes.

7B. Modelo de Actores

Propiedades del actor

Desarrollado por Carl Hewitt (1973), popularizado por Erlang. Similar al pasaje de mensajes.

- **Livianos:** se pueden crear miles en lugar de threads.
- Encapsulan **comportamiento y estado** (privado).
- **Componentes:** dirección (mailbox address) + mailbox (FIFO de mensajes).
- **Aislados:** no comparten memoria. Estado privado solo cambia procesando mensajes.
- Procesan **un mensaje por vez**.
- En sistemas distribuidos, la dirección puede ser remota.
- El actor **supervisor** puede crear actores hijo.

Ciclo de vida (Actix)

Estado	Descripción
Started	Método started(). Contexto disponible.
Running	Estado normal. Puede durar indefinidamente.
Stopping	Al llamar Context::stop, sin referencias externas, o sin objetos en contexto.
Stopped	Estado final.

Formas de enviar mensajes (Actix)

Método	Comportamiento
do_send(M)	Ignora errores. Si mailbox cerrado, descarta. No retorna resultado.
try_send(M)	Inmediato. Si mailbox lleno o cerrado, retorna SendError.
send(M)	Retorna Future con el resultado del handler del mensaje.

Filósofos con Actores — Solución Chandy/Misra

Solución distribuida al problema de los filósofos usando pasaje de mensajes (sin estado compartido).

- Para cada par de filósofos que compiten por un palito, se crea un palito asignado inicialmente al de **ID más bajo** (evita deadlock).
- Cada palito puede estar **sucio** o **limpio**. Inicialmente todos sucios.
- Para comer, un filósofo pide los palitos que no tiene a sus vecinos.
- Si tiene el palito y está **limpio**: lo retiene (vecino queda en espera).
- Si tiene el palito y está **sucio**: lo limpia y lo entrega.
- Al terminar de comer: todos sus palitos quedan **sucios**. Si le habían solicitado alguno, lo limpia y envía.

✓ Esta solución garantiza ausencia de deadlock y starvation.

★ Problemas Clásicos — Pseudocódigo completo

Problema 1: Productor-Consumidor

Productores generan ítems; consumidores los consumen de un buffer compartido.

- Premisas: no consumir lo que no hay; todos los ítems producidos son consumidos; acceso al buffer de a uno; respetar orden FIFO.

Buffer Infinito — con Semáforos

```
sem notEmpty(0) Productor: Consumidor: loop forever loop forever producir d wait(notEmpty) append(d, buffer) d <- take(buffer) signal(notEmpty) consumir(d)
```

Buffer Acotado (tamaño N) — con Semáforos

```
sem notEmpty(0), notFull(N) Productor: Consumidor: loop forever loop forever producir d wait(notEmpty) wait(notFull) d <- take(buffer) append(d, buffer) signal(notFull) signal(notEmpty) consumir(d)
```

Buffer Acotado — con Condvar (Rust)

```
let pair = Arc::new((Mutex::new(VecDeque::new()), Condvar::new())); // Productor: let mut buf = cvar_pair.0.lock().unwrap(); buf = cvar_pair.1.wait_while(buf, |b| b.len() >= N).unwrap(); buf.push_back(item); cvar_pair.1.notify_all(); // Consumidor: let mut buf = cvar_pair.0.lock().unwrap(); buf = cvar_pair.1.wait_while(buf, |b| b.is_empty()).unwrap(); let item = buf.pop_front().unwrap(); cvar_pair.1.notify_all();
```

Problema 2: Barbero Durmiente

Una barbería tiene sala de espera con sillas. Si está vacía, el barbero duerme. Si entra un cliente y el barbero duerme, lo despierta. Si está atendiendo, el cliente espera.

Con Semáforos

```
sem customer_waiting(0) // clientes esperando al barbero sem barber_ready(0) // barbero listo para atender sem haircut_done(0) // corte terminado Barbero: Cliente: loop forever loop forever wait(customer_waiting) signal(customer_waiting) // avisa que llegó signal(barber_ready) wait(barber_ready) // espera al barbero cortar_pelo() sentarse_en_silla() signal(haircut_done) wait(haircut_done) // espera fin de corte
```

Con Condvar (Rust — estructura)

```
// Arc<(Mutex<State>, Condvar)> // State { waiting_customers: usize, barber_sleeping: bool } // Barbero: let mut state = lock.lock().unwrap(); if state.waiting_customers == 0 { state.barber_sleeping = true; } state = cvar.wait_while(state, |s| s.waiting_customers == 0).unwrap(); state.waiting_customers -= 1; state.barber_sleeping = false; // ... cortar pelo ... // Cliente: let mut state = lock.lock().unwrap(); state.waiting_customers += 1; if state.barber_sleeping { cvar.notify_one(); } // esperar fin de corte...
```

Problema 3: Fumadores

3 ingredientes: tabaco, papel, fósforos. 3 fumadores, cada uno con un ingrediente en cantidad infinita. Un agente pone aleatoriamente 2 ingredientes en la mesa. El fumador que tiene el tercero toma los 2 y fuma.

Con Semáforos

```
// Un semáforo por fumador (ingredient_sem[0,1,2]) // sem table_free(1) para que el agente espere Agente: Fumador i: loop forever loop forever wait(table_free) wait(ingredient_sem[i]) ings = random 2 of {0,1,2} fumar() for ing in ings: signal(table_free) signal(ingredient_sem[ing])
```

Con Condvar (Rust — estructura)

```
// Arc<(Mutex<[bool;3]>, Condvar)> // ings[i] = hay ingrediente i en la mesa // Agente: let mut ings = lock.lock().unwrap(); cvar.wait_while(ings, |i| i.iter().any(|x| *x)).unwrap(); // espera mesa libre ings[a] = true; ings[b] = true; cvar.notify_all(); // Fumador i (tiene ingrediente i, necesita los otros dos): let mut ings = lock.lock().unwrap(); ings = cvar.wait_while(ings, |i| !(i[j] &&
```

```
i[k])).unwrap()); // espera sus 2 ins[j] = false; ings[k] = false; cvar.notify_all(); // avisa que la mesa quedó libre
```

Problema 4: Filósofos Comensales

5 filósofos, 5 palitos. Para comer necesita el palito de su izquierda y el de su derecha. Si todos agarran el izquierdo simultáneamente → deadlock.

Solución: Romper simetría (el N-ésimo toma al revés)

```
// Filósofo i: if i == N-1: first = chopstick[next] // N-ésimo agarra DERECHA primero second = chopstick[i] else: first = chopstick[i] // los demás agarran IZQUIERDA primero second = chopstick[next] loop: pensar() acquire(first) // semáforo binario por palito acquire(second) comer() release(first) release(second)
```

✓ Romper la simetría garantiza ausencia de deadlock. Un filósofo siempre podrá comer.

Solución: Mozo mediador (channel/actor)

- Un mozo controla el acceso a los palitos. Los filósofos le piden al mozo, no directamente entre sí.
- El mozo puede implementar políticas de fairness (ej: no dar un palito si el filósofo adyacente lleva mucho tiempo esperando).

Problema 5: Lector-Escritor

Un estado compartido. Múltiples procesos leen (sin conflicto entre sí) o escriben (exclusivo). Mientras un escritor escribe, nadie puede leer ni escribir.

Normal (sin preferencia) — con Condvar

```
// Arc<(Mutex<State>, Condvar)> // State { readers: u32, writing: bool } // Lector: let mut s = cvar.wait_while(lock.lock().unwrap(), |s| s.writing).unwrap(); s.readers += 1; drop(s); // ... leer ... lock.lock().unwrap().readers -= 1; cvar.notify_all(); // Escritor: let mut s = cvar.wait_while(lock.lock().unwrap(), |s| s.writing || s.readers > 0).unwrap(); s.writing = true; drop(s); // ... escribir ... lock.lock().unwrap().writing = false; cvar.notify_all();
```

Con preferencia de escritura (writer preference)

Agregar campo writers: u32 al estado. Un lector espera si writing || writers > 0. El escritor incrementa writers antes de esperar la sección crítica, garantizando que nuevos lectores no entren mientras hay escritores esperando.

✓ Writer preference resuelve starvation de escritores pero puede causar starvation de lectores.

✓ Checklist Rápido Pre-Parcial

Conceptos que sí o sí tenés que saber definir

- Instrucción atómica, proceso, programa concurrente.
- Safety vs Liveness. Ejemplos de cada una.
- Exclusión mutua, deadlock, starvation, fairness.
- Sección crítica: las 3 especificaciones.
- Semáforo: definición formal, invariante, wait/signal.
- Diferencias semáforo vs condvar (la tabla de 4 filas).
- Monitor: componentes, exclusión mutua implícita.
- Futures: qué es poll, Pending vs Ready.
- Fork-Join: determinismo, work stealing.
- Actor: mailbox, aislamiento, do_send vs try_send vs send.
- Red de Petri: función de marca, reglas de disparo.

Código que tenés que poder escribir en el parcial

- Arc> — leer y escribir con guards RAI.
- Arc<(Mutex, Condvar)> — patrón wait_while / notify_all.
- Semáforos: Productor-Consumidor buffer infinito y acotado.
- Semáforos: Barbero, Fumadores, Filósofos.
- thread::spawn + join, Rayon par_iter, Crossbeam scope.
- mpsc::channel — send / rcv.
- async fn + await, block_on.

Errores comunes a evitar

- Usar if en lugar de while al esperar en una condvar (spurious wakeups).
- Olvidar que signal() del semáforo siempre tiene efecto (V++), distinto de signalC().
- Confundir Rc (no thread-safe) con Arc (thread-safe).
- No liberar el lock antes de notificar en condvar puede causar deadlock.
- En Fork-Join: las tareas NO deben bloquearse excepto para esperar subtareas.
- Orden de wait() en Filósofos: si todos agarran primero izquierda → deadlock.